

MEMORY MANAGEMENT

Module 5

BACKGROUND

- ❑ Memory consists of a large array of bytes, each with its own address.
- ❑ The CPU fetches instructions from memory according to the value of the program counter.
- ❑ The memory unit sees only a stream of memory addresses; it does not know how they are generated (by the instruction counter, indexing, indirection, literal addresses, and so on) or what they are for (instructions or data).
- ❑ Focuses is the sequence of memory addresses generated by the running program.

HARDWARE

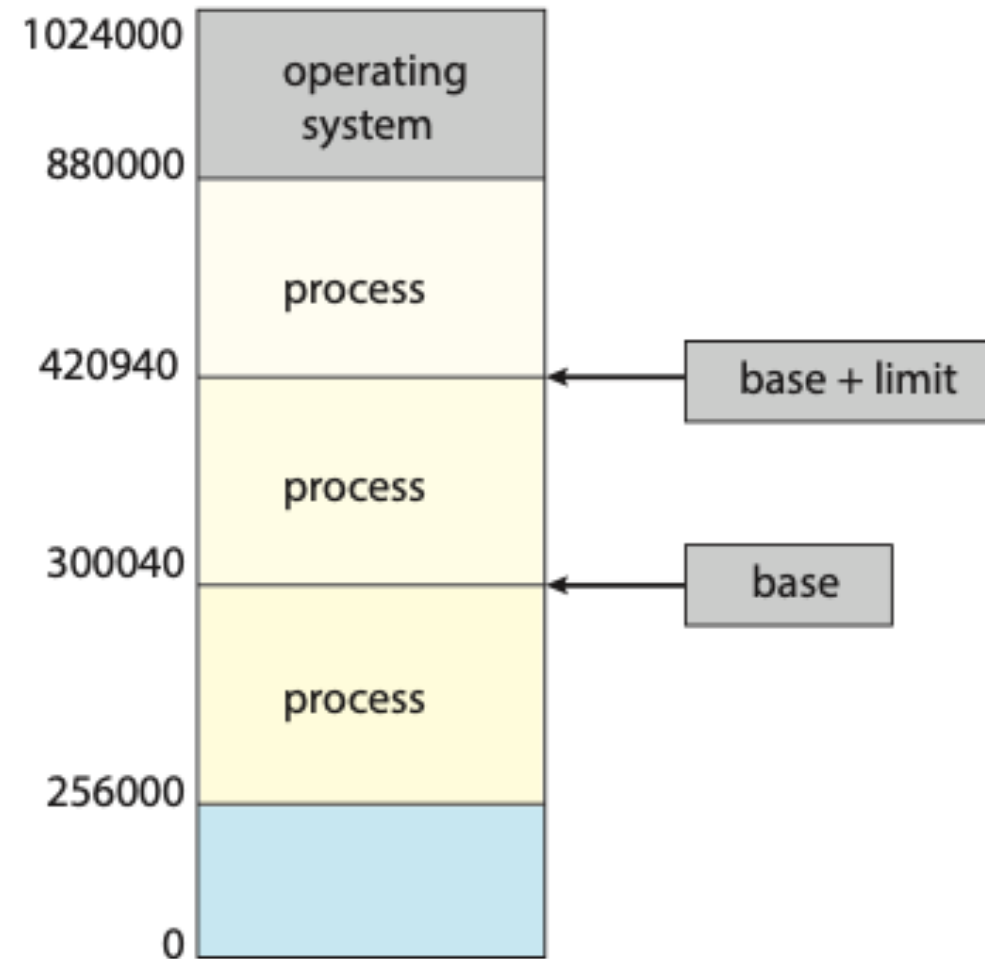
- ❑ Main memory and the registers built into each processing core are the only general-purpose storage that the CPU can access directly.
- ❑ Any instructions in execution, and any data being used by the instructions, must be in one of these direct-access storage devices. If the data are not in memory, they must be moved there before the CPU can operate on them.
- ❑ Registers that are built into each CPU core are generally accessible within one cycle of the CPU clock.
- ❑ CPU cores can decode instructions and perform simple operations on register contents at the rate of one or more operations per clock tick.
- ❑ same cannot be said of main memory, which is accessed via a transaction on the memory bus

HARDWARE – MEMORY PROTECTION

- ❑ For proper system operation, we must protect the operating system from access by user processes, as well as protect user processes from one another.
- ❑ This protection must be provided by the hardware, because the operating system doesn't usually intervene between the CPU and its memory accesses.
- ❑ Make sure that each process has a separate memory space. Separate per-process memory space protects the processes from each other
- ❑ To separate memory spaces, needed the ability to determine the range of legal addresses that the process may access and to ensure that the process can access only these legal addresses.
- ❑ Provide this protection by using two registers, usually a base and a limit.

HARDWARE – MEMORY PROTECTION

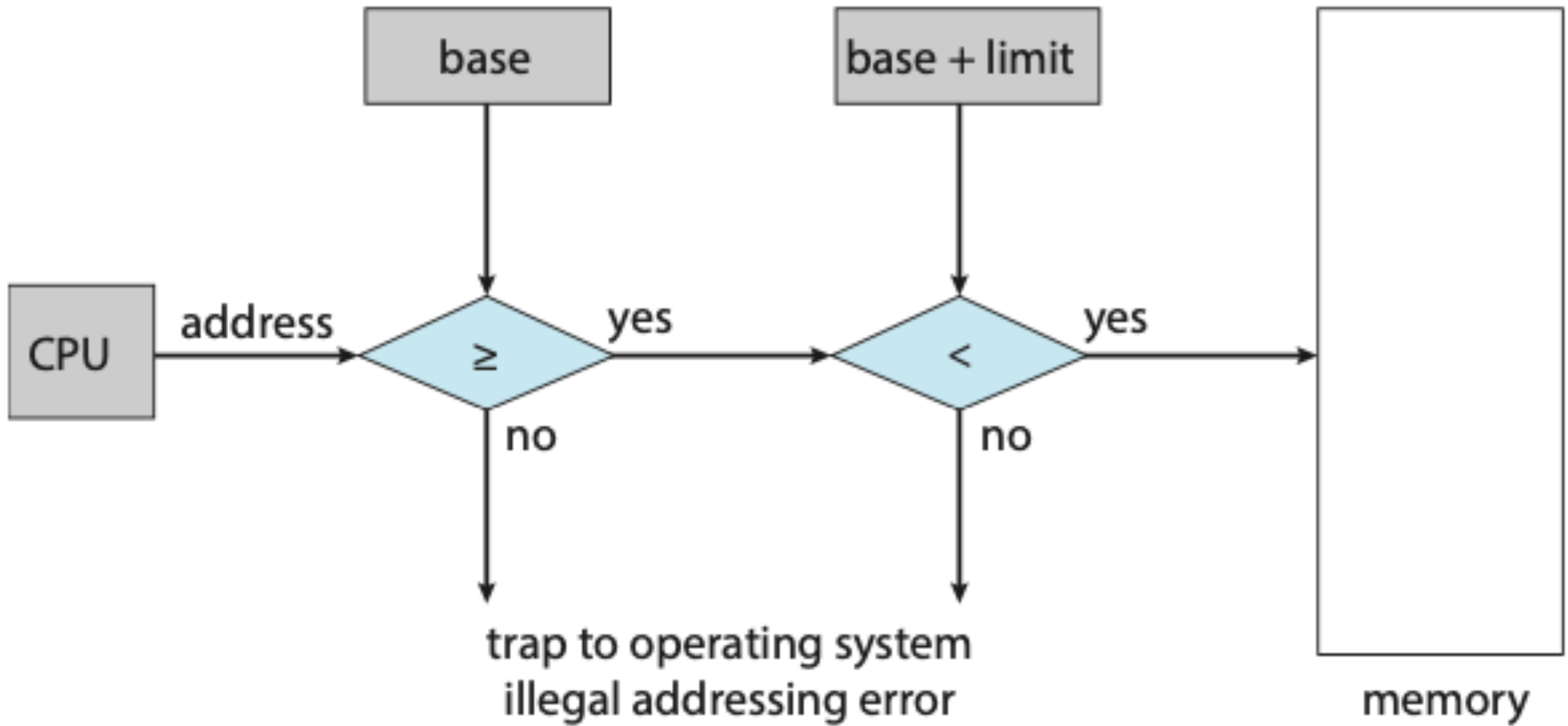
- ❑ The base register holds the smallest legal physical memory address; the limit register specifies the size of the range.



HARDWARE – MEMORY PROTECTION

- ❑ Protection of memory space is accomplished by having the CPU hardware compare every address generated in user mode with the registers
- ❑ Any attempt by a program executing in user mode to access operating-system memory or other users' memory results in a trap to the operating system
- ❑ The base and limit registers can be loaded only by the operating system
- ❑ Since privileged instructions can be executed only in kernel mode, and since only the operating system executes in kernel mode, only the operating system can load the base and limit registers.
- ❑ This scheme allows the operating system to change the value of the registers but prevents user programs from changing the registers' contents.

HARDWARE – MEMORY PROTECTION



HARDWARE – MEMORY ADDRESS BINDING

- ❑ A program resides on a disk as a binary executable file.
- ❑ To run, the program must be brought into memory and placed within the context of a process
- ❑ As the process executes, it accesses instructions and data from memory.
- ❑ Addresses in the source program are generally symbolic (Like variables)
- ❑ A compiler typically binds these symbolic addresses to relocatable addresses
- ❑ The linker or loader in turn binds the relocatable addresses to absolute addresses
- ❑ Each binding is a mapping from one address space to another.

HARDWARE – MEMORY ADDRESS BINDING

❑ **Compile time Address Binding**

- ❑ If you know at compile time where the process will reside in memory, then absolute code can be generated.
- ❑ if you know that a user process will reside starting at location R, then the generated compiler code will start at that location and extend up from there.
- ❑ If the starting location changes, then it will be necessary to recompile this code.

HARDWARE – MEMORY ADDRESS BINDING

❑ Load time Address Binding

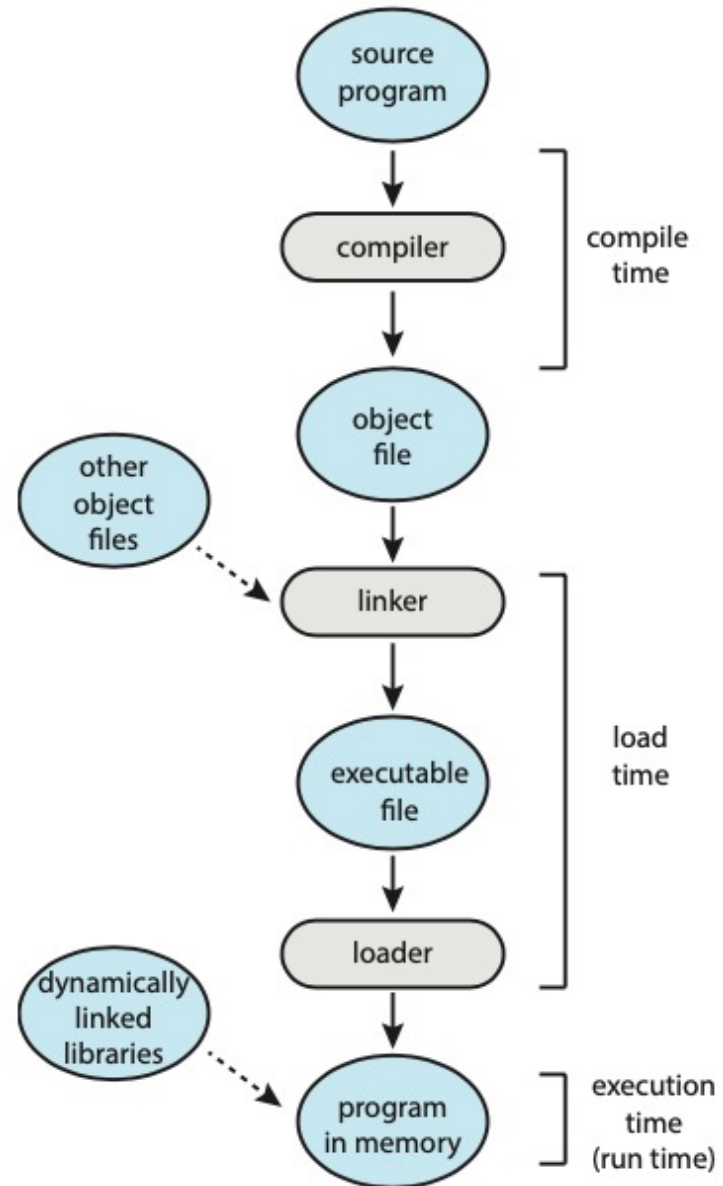
- ❑ If it is not known at compile time where the process will reside in memory, then the compiler must generate relocatable code.
- ❑ In this case, final binding is delayed until load time.
- ❑ If the starting address changes, we need only reload the user code to incorporate this changed value.

HARDWARE – MEMORY ADDRESS BINDING

❑ **Execution time Address Binding**

- ❑ If the process can be moved during its execution from one memory segment to another, then binding must be delayed until run time.

HARDWARE – MEMORY ADDRESS BINDING



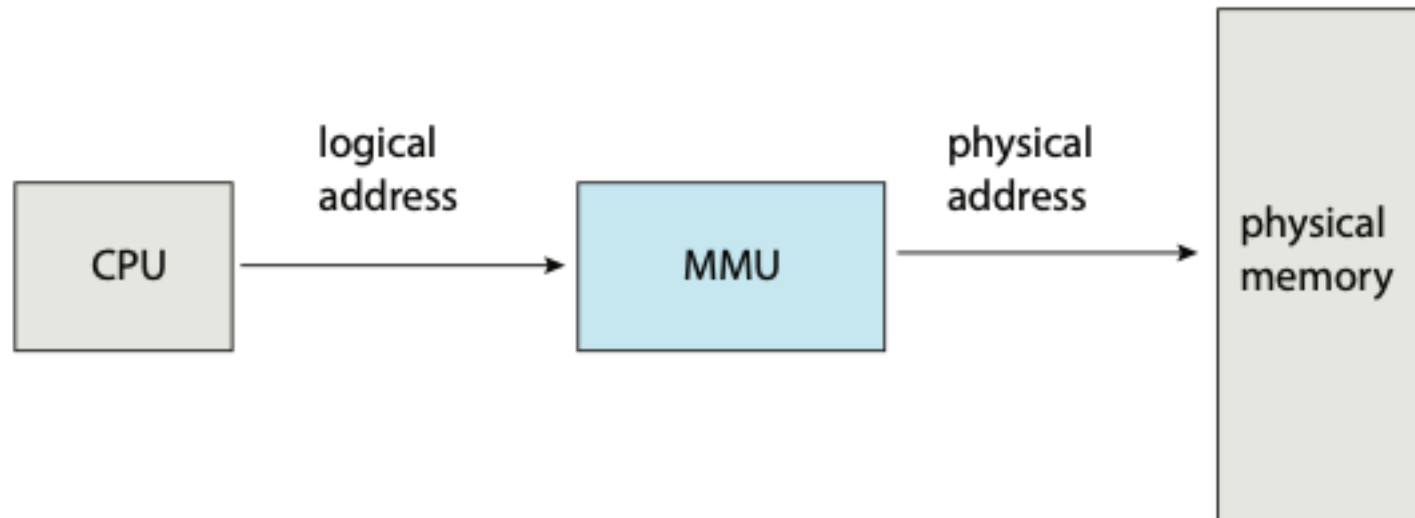
MEMORY ADDRESSES

❑ Logical Address

- ❑ An address generated by the CPU.

❑ Physical Address

- ❑ An address seen by the memory unit—that is, the one loaded into the memory-address register of the memory



MEMORY ADDRESSES

- ❑ Binding addresses at either compile or load time generates identical logical and physical addresses.
- ❑ the execution-time address-binding scheme results in differing logical and physical addresses.
- ❑ The set of all logical addresses generated by a program is a logical address space.
- ❑ The set of all physical addresses corresponding to these logical addresses is a physical address space.
- ❑ The run-time mapping from virtual to physical addresses is done by a hardware device called the **memory-management unit (MMU)**.
- ❑ The use of logical addresses provides a layer of abstraction that allows processes to access memory without knowing the physical memory location.

MEMORY MANAGEMENT UNIT

- ❑ The Memory Management Unit (MMU) is a hardware component that translates virtual addresses to physical addresses.
- ❑ The **relocation register** is a special register that is used to help the MMU perform this translation.
- ❑ When a process is running, the CPU generates virtual addresses.
- ❑ These virtual addresses are not the same as the physical addresses that are stored in memory.
- ❑ The MMU uses the relocation register to add a fixed offset to each virtual address.
- ❑ This offset is the difference between the base address of the process's virtual address space and the start of physical memory.
- ❑ The relocation register is used to protect different processes from each other. Each process has its own relocation register, and the MMU will not allow a process to access memory that belongs to another process.

CONTIGUOUS MEMORY ALLOCATION

- ❑ The main memory must accommodate both the operating system and the various user processes.
- ❑ The memory is usually divided into two partitions: one for the operating system and one for the user processes.
- ❑ Usually, several user processes reside in memory at the same time. Thus, need to consider how to allocate available memory to the processes that are waiting to be brought into memory.

CONTIGUOUS MEMORY ALLOCATION

- ❑ Each process is contained in a single section of memory that is contiguous to the section containing the next process.
- ❑ Memory Allocation: **Fixed/Static Partitioning**
 - ❑ Allocate memory to processes in contiguous blocks
 - ❑ A process is allocated a single, contiguous block of memory.
 - ❑ Is simpler to implement.
 - ❑ Lead to fragmentation, which occurs when there are unused gaps of memory between allocated blocks.
 - ❑ Fragmentation can make it difficult to allocate memory to new processes, and it can also reduce the amount of available memory.
 - ❑ Demerits
 - ❑ Can lead to fragmentation
 - ❑ Can make it difficult to allocate memory to new processes
 - ❑ Can reduce the amount of available memory

CONTIGUOUS MEMORY ALLOCATION

- ❑ Each process is contained in a single section of memory that is contiguous to the section containing the next process.
- ❑ Memory Allocation: **Dynamic/Variable Partitioning**
 - ❑ To assign processes to variably sized partitions in memory, where each partition may contain exactly one process.
 - ❑ The operating system keeps a table indicating which parts of memory are available and which are occupied.
 - ❑ Initially, all memory is available for user processes and is considered one large block of available memory, a hole.

CONTIGUOUS MEMORY ALLOCATION

- ❑ The memory blocks available comprise a set of holes of various sizes scattered throughout memory.
- ❑ When a process arrives and needs memory, the system searches the set for a hole that is large enough for this process.
- ❑ If the hole is too large, it is split into two parts.
- ❑ One part is allocated to the arriving process; the other is returned to the set of holes.
- ❑ When a process terminates, it releases its block of memory, which is then placed back in the set of holes.
- ❑ If the new hole is adjacent to other holes, these adjacent holes are merged to form one larger hole.

CONTIGUOUS MEMORY ALLOCATION

❑ Problem:

- ❑ Dynamic storage- allocation problem

- ❑ How to satisfy a request of size n from a list of free **holes**.

❑ Important Terms:

- ❑ Holes/Hole – Block of available memory.

CONTIGUOUS MEMORY ALLOCATION SCHEMES

❑ **First fit.**

- ❑ Allocate the first hole that is big enough.
- ❑ Searching can start either at the beginning of the set of holes or at the location where the previous first-fit search ended (**Next Fit**).
- ❑ Stop searching as soon as we find a free hole that is large enough.

❑ Merit:

- ❑ Search for a suitable block of memory can be performed quickly and easily.

❑ Demerit

- ❑ Poor performance in situations where the memory is highly fragmented
- ❑ Can lead to poor memory utilization.

CONTIGUOUS MEMORY ALLOCATION SCHEMES

❑ **Best fit.**

- ❑ Allocate the smallest hole that is big enough.
- ❑ Search the entire list, unless the list is ordered by size.
- ❑ This strategy produces the smallest leftover hole.

❑ **Merit:**

- ❑ Improved memory utilization, as it allocates the smallest block of memory that is sufficient to accommodate the memory request from the process.
- ❑ Reduce memory fragmentation, as it tends to allocate smaller blocks of memory that are less likely to become fragmented.

❑ **Demerit:**

- ❑ Increased computational overhead, as the search for the best-fit block of memory.
- ❑ Result in increased fragmentation, as it may leave smaller blocks of memory scattered.

CONTIGUOUS MEMORY ALLOCATION SCHEMES

❑ **Worst fit.**

- ❑ Allocate the largest hole.
- ❑ Search the entire list, unless it is sorted by size.
- ❑ This strategy produces the largest leftover hole, which may be more useful than the smaller leftover hole from a best-fit approach.

❑ Demerit:

- ❑ Slow process and Traverses the entire memory.

CONTIGUOUS MEMORY ALLOCATION SCHEMES - PROBLEM

□ Suppose you have a memory with the following free blocks: 12KB, 6KB, 14KB, 19KB, 21KB. Four processes arrive that require 10KB, 4KB, 7KB, and 18KB of memory, respectively. Using the different allocation techniques, how the process would be allocated memory.

CONTIGUOUS MEMORY ALLOCATION SCHEMES - SOLUTION

First Fit Allocation Scheme.

All Blocks are empty at the beginning.

PROCESS No.	PROCESS SIZE	BLOCK No.	BLOCK SIZE	AVAILABILITY STATUS	CAN OCCUPY PROCESS ?	BLOCK STATUS	MEMORY WASTAGE
P1	10KB	BLOCK 1	12KB	AVAILABLE	CHECK $10 < 12$	P1 ALLOTTED	$12 - 10 = 2\text{KB}$
P2	4KB	BLOCK 1	12KB	OCCUPIED	No	OCCUPIED	-
P2	4KB	BLOCK 2	6KB	AVAILABLE	CHECK $4 < 6$	P2 ALLOTTED	$6 - 4 = 2\text{KB}$
P3	7KB	BLOCK 1	12KB	OCCUPIED	No	OCCUPIED	-
P3	7KB	BLOCK 2	6KB	OCCUPIED	No	OCCUPIED	-
P3	7KB	BLOCK 3	14KB	AVAILABLE	CHECK $7 < 14$	P3 ALLOTTED	$14 - 7 = 7\text{KB}$
P4	18KB	BLOCK 1	12KB	OCCUPIED	No	OCCUPIED	-
P4	18KB	BLOCK 2	6KB	OCCUPIED	No	OCCUPIED	-
P4	18KB	BLOCK 3	14KB	OCCUPIED	No	OCCUPIED	-
P4	18KB	BLOCK 4	19 KB	AVAILABLE	CHECK $18 < 19$	P4 ALLOTTED	$19 - 18 = 1\text{KB}$
		BLOCK 5	21KB	AVAILABLE		EMPTY	

CONTIGUOUS MEMORY ALLOCATION SCHEMES - SOLUTION

Next Fit Allocation Scheme.

All Blocks are empty at the begining.

PROCESS No.	PROCESS SIZE	BLOCK No.	BLOCK SIZE	AVAILABILITY STATUS	CAN OCCUPY PROCESS ?	BLOCK STATUS	MEMORY WASTAGE
P1	10KB	BLOCK 1	12KB	AVAILABLE	CHECK $10 < 12$	P1 ALLOTTED	$12 - 10 = 2\text{KB}$
P2	4KB	BLOCK 2	6KB	AVAILABLE	CHECK $4 < 6$	P2 ALLOTTED	$6 - 4 = 2\text{KB}$
P3	7KB	BLOCK 3	14KB	AVAILABLE	CHECK $7 < 14$	P3 ALLOTTED	$14 - 7 = 7\text{KB}$
P4	18KB	BLOCK 4	19 KB	AVAILABLE	CHECK $18 < 19$	P4 ALLOTTED	$19 - 18 = 1\text{KB}$
		BLOCK 5	21KB	AVAILABLE		EMPTY	

CONTIGUOUS MEMORY ALLOCATION SCHEMES - SOLUTION

Best Fit Allocation Scheme.

All Blocks are empty at the beginning.

PROCES S No.	PROCESS SIZE	BLOCK No.	BLOCK SIZE	AVAILABILITY STATUS	CAN OCCUPY PROCESS ?	MEMORY WASTAGE	FIND MINIMUM WASTAGE BLOCK	BLOCK STATUS
P1	10KB	BLOCK 1	12KB	AVAILABLE	CHECK $10 < 12$	$12 - 10 = 2\text{KB}$	$2 < 4 < 9 < 11$ BLOCK 1	P1 ALLOTTED
		BLOCK 2	6KB	AVAILABLE	CHECK $10 < 6$	NOT FITTING		EMPTY
		BLOCK 3	14KB	AVAILABLE	CHECK $10 < 14$	$14 - 10 = 4\text{KB}$		EMPTY
		BLOCK 4	19 KB	AVAILABLE	CHECK $10 < 19$	$19 - 10 = 9\text{KB}$		EMPTY
		BLOCK 5	21KB	AVAILABLE	CHECK $21 < 10$	$21 - 10 = 11\text{KB}$		EMPTY

CONTIGUOUS MEMORY ALLOCATION SCHEMES - SOLUTION

Best Fit Allocation Scheme.

All Blocks are empty at the beginning.

PROCES S No.	PROCESS SIZE	BLOCK No.	BLOCK SIZE	AVAILABILITY STATUS	CAN OCCUPY PROCESS ?	MEMORY WASTAGE	FIND MINIMUM WASTAGE BLOCK	BLOCK STATUS
P2	4KB	BLOCK 1	12KB	UNAVAILABLE			2 < 10 < 15 < 17 BLOCK 2	P1 ALLOTTED
		BLOCK 2	6KB	AVAILABLE	CHECK 4 < 6	6 – 4 = 2KB		P2 ALLOTTED
		BLOCK 3	14KB	AVAILABLE	CHECK 4 < 14	14 – 4 = 10KB		EMPTY
		BLOCK 4	19 KB	AVAILABLE	CHECK 4 < 19	19 – 4 = 15KB		EMPTY
		BLOCK 5	21KB	AVAILABLE	CHECK 4 < 21	21 – 4 = 17KB		EMPTY

CONTIGUOUS MEMORY ALLOCATION SCHEMES - SOLUTION

Best Fit Allocation Scheme.

All Blocks are empty at the beginning.

PROCES S No.	PROCESS SIZE	BLOCK No.	BLOCK SIZE	AVAILABILITY STATUS	CAN OCCUPY PROCESS ?	MEMORY WASTAGE	FIND MINIMUM WASTAGE BLOCK	BLOCK STATUS
P3	7KB	BLOCK 1	12KB	UNAVAILABLE			7 < 12 < 14 BLOCK 3	P1 ALLOTTED
		BLOCK 2	6KB	UNAVAILABLE				P2 ALLOTTED
		BLOCK 3	14KB	AVAILABLE	CHECK 7 < 14	14 – 7 = 7KB		P3 ALLOTTED
		BLOCK 4	19 KB	AVAILABLE	CHECK 7 < 19	19 – 7 = 12KB		EMPTY
		BLOCK 5	21KB	AVAILABLE	CHECK 7 < 21	21 – 7 = 14KB		EMPTY

CONTIGUOUS MEMORY ALLOCATION SCHEMES - SOLUTION

Best Fit Allocation Scheme.

All Blocks are empty at the beginning.

PROCES S No.	PROCESS SIZE	BLOCK No.	BLOCK SIZE	AVAILABILITY STATUS	CAN OCCUPY PROCESS ?	MEMORY WASTAGE	FIND MINIMUM WASTAGE BLOCK	BLOCK STATUS
P4	18KB	BLOCK 1	12KB	UNAVAILABLE			1 < 3 BLOCK 4	P1 ALLOTTED
		BLOCK 2	6KB	UNAVAILABLE				P2 ALLOTTED
		BLOCK 3	14KB	UNAVAILABLE				P3 ALLOTTED
		BLOCK 4	19 KB	AVAILABLE	CHECK 18 < 19	19 – 18 = 1KB		P4 ALLOTTED
		BLOCK 5	21KB	AVAILABLE	CHECK 7 < 21	21 – 18 = 3KB		EMPTY

CONTIGUOUS MEMORY ALLOCATION SCHEMES - SOLUTION

Worst Fit Allocation Scheme.

All Blocks are empty at the beginning.

PROCES S No.	PROCESS SIZE	BLOCK No.	BLOCK SIZE	AVAILABILITY STATUS	CAN OCCUPY PROCESS ?	MEMORY WASTAGE	FIND MINIMUM WASTAGE BLOCK	BLOCK STATUS
P1	10KB	BLOCK 1	12KB	AVAILABLE	CHECK $10 < 12$	$12 - 10 = 2\text{KB}$	$11 > 9 > 4 > 2$ BLOCK 5	EMPTY
		BLOCK 2	6KB	AVAILABLE	CHECK $10 < 6$	NOT FITTING		EMPTY
		BLOCK 3	14KB	AVAILABLE	CHECK $10 < 14$	$14 - 10 = 4\text{KB}$		EMPTY
		BLOCK 4	19 KB	AVAILABLE	CHECK $10 < 19$	$19 - 10 = 9\text{KB}$		EMPTY
		BLOCK 5	21KB	AVAILABLE	CHECK $21 < 10$	$21 - 10 = 11\text{KB}$		P1 ALLOTTED

CONTIGUOUS MEMORY ALLOCATION SCHEMES - SOLUTION

Worst Fit Allocation Scheme.

All Blocks are empty at the beginning.

PROCES S No.	PROCESS SIZE	BLOCK No.	BLOCK SIZE	AVAILABILITY STATUS	CAN OCCUPY PROCESS ?	MEMORY WASTAGE	FIND MINIMUM WASTAGE BLOCK	BLOCK STATUS
P2	4KB	BLOCK 1	12KB	AVAILABLE	CHECK $4 < 12$	$12 - 4 = 8\text{KB}$	$15 > 10 > 8 > 2$ BLOCK 4	EMPTY
		BLOCK 2	6KB	AVAILABLE	CHECK $4 < 6$	$6 - 4 = 2\text{KB}$		EMPTY
		BLOCK 3	14KB	AVAILABLE	CHECK $4 < 14$	$14 - 4 = 10\text{KB}$		EMPTY
		BLOCK 4	19 KB	AVAILABLE	CHECK $4 < 19$	$19 - 4 = 15\text{KB}$		P2 ALLOTTED
		BLOCK 5	21KB	UNAVAILABLE				P1 ALLOTTED

CONTIGUOUS MEMORY ALLOCATION SCHEMES - SOLUTION

Worst Fit Allocation Scheme.

All Blocks are empty at the beginning.

PROCES S No.	PROCESS SIZE	BLOCK No.	BLOCK SIZE	AVAILABILITY STATUS	CAN OCCUPY PROCESS ?	MEMORY WASTAGE	FIND MINIMUM WASTAGE BLOCK	BLOCK STATUS
P3	7KB	BLOCK 1	12KB	AVAILABLE	CHECK $7 < 12$	$12 - 7 = 5\text{KB}$	$7 > 5$ BLOCK 3	
		BLOCK 2	6KB	AVAILABLE	CHECK $7 < 6$	NOT FITTING		
		BLOCK 3	14KB	AVAILABLE	CHECK $7 < 14$	$14 - 7 = 7\text{KB}$		P3 ALLOTTED
		BLOCK 4	19 KB	UNAVAILABLE				P2 ALLOTTED
		BLOCK 5	21KB	UNAVAILABLE				P1 ALLOTTED

CONTIGUOUS MEMORY ALLOCATION SCHEMES - SOLUTION

Best Fit Allocation Scheme.

All Blocks are empty at the beginning.

PROCES S No.	PROCESS SIZE	BLOCK No.	BLOCK SIZE	AVAILABILITY STATUS	CAN OCCUPY PROCESS ?	MEMORY WASTAGE	FIND MINIMUM WASTAGE BLOCK	BLOCK STATUS
P4	18KB	BLOCK 1	12KB	AVAILABLE	CHECK $18 < 12$	NOT FITTING	NO LARGE BLOCK AVAILABLE, REJECT THE PROCESS P4 OR MAKE IT TO WAIT P4 REMAINS UNALLOCATED	EMPTY
		BLOCK 2	6KB	AVAILABLE	CHECK $18 < 6$	NOT FITTING		EMPTY
		BLOCK 3	14KB	UNAVAILABLE				P3 ALLOTTED
		BLOCK 4	19 KB	UNAVAILABLE				P2 ALLOTTED
		BLOCK 5	21KB	UNAVAILABLE				P1 ALLOTTED

FRAGMENTATION

- ❑ **External Fragmentation** – Total memory space exists to satisfy a request, but it is not contiguous.
- ❑ **Internal Fragmentation** – Allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used.
- ❑ First fit analysis reveals that given N blocks allocated, $0.5 N$ blocks lost to fragmentation.
 - ❑ $1/3$ may be unusable -> 50-percent rule
 - ❑ This rule highlights the issue of external fragmentation that can occur when using the first-fit allocation technique.

FRAGMENTATION

- ❑ Reduce external fragmentation by **compaction**
 - ❑ Shuffle memory contents to place all free memory together in one large block.
 - ❑ Compaction is possible only if relocation is dynamic, and is done at execution time
 - ❑ I/O problem
 - ❑ Latch job in memory while it is involved in I/O
 - ❑ Do I/O only into OS buffers
 - ❑ Now consider that backing store has same fragmentation problems

PAGING

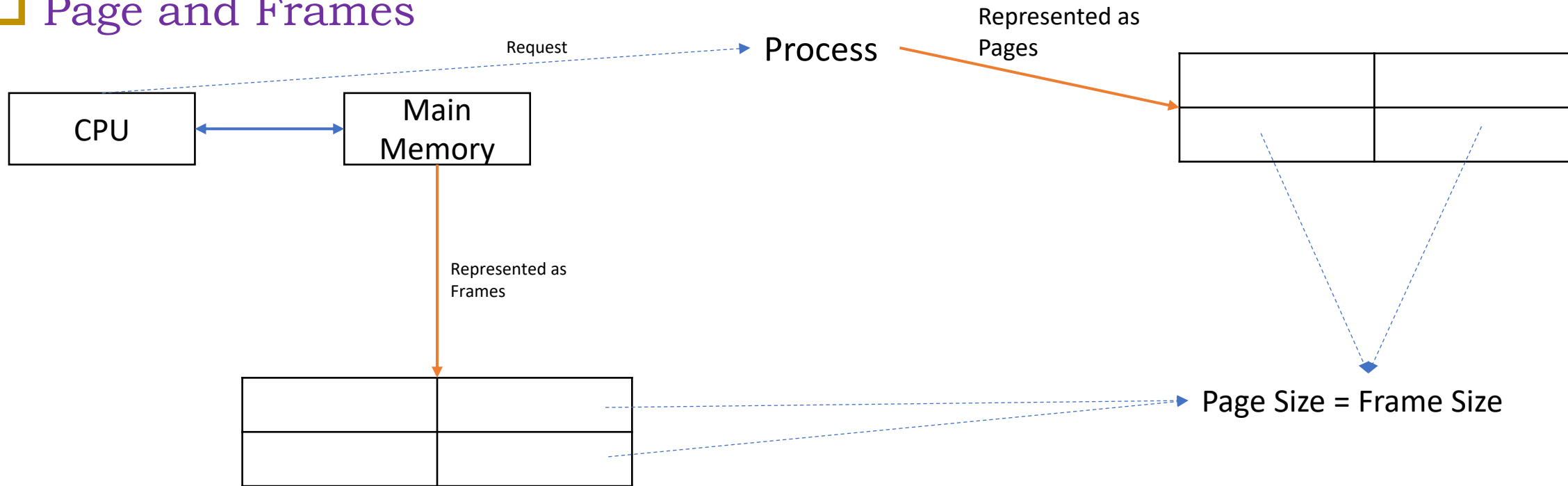
- ❑ It is a memory management technique used by operating systems to manage the allocation of memory to processes.
- ❑ It permits the physical address space of a process to be noncontiguous or in other words eliminates the need for contiguous allocation of physical memory.
- ❑ It allows viewing memory spaces that physically lie at different locations in the hardware to be logically viewed as contiguous.
- ❑ Facilitated by mapping of physical and virtual memory that is done by hardware component called memory management unit (MMU).

PAGING - IDEA

- ❑ Divide each process into fixed-size units called **pages**.
- ❑ The main memory is also divided into fixed-size blocks called **frames**
- ❑ Pages are then loaded into available frames in the main memory as needed.
- ❑ When a process needs to access data that is not currently in main memory, the operating system will bring the required page from secondary storage into an available frame in main memory.

PAGING – TERMINOLOGIES AND CONCEPT

❑ Page and Frames

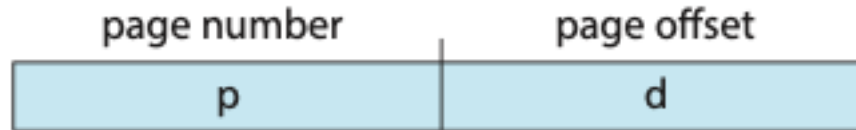


❑ Pages are Logical

❑ Frames are Physical

PAGING – BASIC METHOD

- Every address generated by the CPU is divided into two parts: a page number (p) and a page offset (d).



- The page number is used as an index into a per-process page table.
- The page table contains the base address of each frame in physical memory, and the offset is the location in the frame being referenced.
- The base address of the frame is combined with the page offset to define the physical memory address.

PAGING - TERMINOLOGIES

❑ Logical View

- ❑ **Logical Space Available** – The total space available in logical system.
- ❑ **Logical Address** – This is also known as Virtual Address and is directly generated by the CPU of the system.

❑ Example:

- ❑ If Logical address is 21bit then $\rightarrow$ Logical Address space = 2^{21} which is $2^3 * 2^{20}$. Thus, 2M words
- ❑ If Logical address is 23bit then $\rightarrow$ Logical Address space = 2^{23} which is $2^3 * 2^{20}$. Thus, 8M words.

PAGING - TERMINOLOGIES

❑ Remember

- ❑ 1 byte = 2^3 bits
- ❑ 1KB = 2^{10} bytes
- ❑ 1MB = 2^{20} bytes
- ❑ 1GB = 2^{30} bytes.

❑ Physical View

- ❑ **Physical Space Available** – All possible set of physical store locations
- ❑ **Physical Address** – This is true address as seen where information is stored in the hardware.

PAGING - TERMINOLOGIES

❑ Frames

- ❑ They are basically the sliced up physical memory blocks of equal size.

Example : 512kb of memory can be divided into 4 parts for 128kb each.

No. of Frames = Physical address space / Frame size

❑ Pages

- ❑ Pages are sliced up logical memory blocks of equal size.

Generally, always the page size should be equal to frame size.

No. of pages = Logical address space / Page size

PAGING - TERMINOLOGIES

❑ Logical Address

❑ Page address is called logical address and is a combination of by page number (denoted as 'p') and offset (denoted as 'd').

Note: Page number sometimes is also called as VPN(Virtual Page Number).

$$\text{Logical Address} = \text{Page number} + \text{page offset}$$

Page Number(p)- Number of bits required to represent the pages in Logical Address Space.

Page offset(d) – Number of bits required to represent particular word in a page or page size of Logical Address Space

PAGING - TERMINOLOGIES

❑ Physical Address

- ❑ Frame address is called physical address and is a combination of a frame number (denoted as 'f') and the offset (denoted as 'd').

Note: Frame number is also sometimes referred as PFN (Physical Frame Number).

Physical Address = Frame number + page offset

Frame Number(f) – Number of bits required to represent the pages in Physical Address Space

Frame offset(d) – Number of bits required to represent particular word in a page or page size of Physical Address Space

PAGING – TERMINOLOGIES - PROBLEM

- ❑ Consider a machine with 64 MB physical memory and a 32-bit virtual address space. If the page size is 4KB, what is the approximate size of the page table?